

Writing the Workflow Spec I: Objective, Inputs, and Output Contract

FRIDAY AI Education — Episode 032 · 2026-07-18 · Printable Companion

AI-Native Operator — Lesson 18 of 37 · Workflow Specification, Part 1 of 2

Abstract

Before you ask an AI system to run a workflow, you should be able to write down four things: what the workflow is for, what it must not attempt, what it is allowed to consume, and what its output must look like. This paper introduces the first half of the workflow spec — objective, non-goals, allowed inputs, and the output contract, tied together by acceptance criteria. The argument is simple: most of the workflow failures studied in the previous two lessons (bad inputs, false confidence, unsafe action, drift) are not model failures. They are specification failures that the model merely executed. A spec is cheap, it is written in prose, and it is the single highest-leverage artifact in workflow automation — because it converts "the AI got it wrong" into "the spec was wrong," which is a problem you can actually fix. Part II of this unit covers the runtime half of the spec: gates, failure handling, and the definition of done.

1. The Bridge: Failure Modes Were Spec Gaps Wearing Costumes

The last two lessons in this arc catalogued how automated workflows fail. Bad inputs and false confidence: the workflow consumed something stale, wrong, or out of scope, and produced output that looked authoritative anyway. Unsafe action and drift: the workflow did something it shouldn't have, or slowly wandered away from its purpose while everyone assumed it was still on task.

Here is the observation that motivates this entire two-part unit: **almost every failure in that catalogue traces back to a question nobody answered before the workflow was built.**

- The workflow ingested a stale price feed → nobody wrote down which inputs were allowed and how fresh they had to be.
- The workflow confidently summarized a document it never actually retrieved → nobody defined what the output must contain (citations, provenance) for it to count as valid.
- The workflow sent an email it should have drafted → nobody wrote down the non-goal "this workflow does not send anything."
- The workflow drifted from "research targets" to "write outreach copy" over six weeks of incremental prompt edits → nobody wrote down the objective, so there was nothing to drift *from*.

Debugging those failures after the fact is expensive. Preventing them costs one page of prose. That page is the workflow spec, and today's lesson covers its front half: what the workflow is for and what its interface looks like. The next lesson covers the back half: what happens at runtime — gates, failure handling, and how the workflow knows it is done.

One framing note before we start. A workflow spec is not documentation and it is not a prompt. Documentation describes a system that already exists. A prompt is an implementation detail — one of several possible ways to

realize a spec. The spec comes first, exists independently of any particular model or tool, and outlives both. When you swap models next year, the spec doesn't change. That independence is precisely what makes it valuable.

2. Why the Spec Comes Before the Automation

There is a tempting shortcut that nearly every operator takes at least once: skip the spec, open a chat window, describe the task conversationally, iterate on the output until it looks right, then paste the winning prompt into a scheduled job and call it a workflow.

This works, briefly. Then it fails in a specific and predictable way: **the workflow's actual behavior becomes the only definition of its intended behavior**. When the output degrades — new model version, new input distribution, a prompt edit with side effects — you have no reference point. You cannot say the workflow is broken, because you never said what working meant. You can only say it *feels* different. Organizations run on "feels different" for a surprisingly long time, which is how drift compounds.

The deeper reason the spec comes first is about delegation, and it applies equally to humans and machines. When you hand a workflow to a junior employee, you implicitly answer four questions: what outcome you want, what's out of bounds, what materials they should work from, and what the deliverable looks like. Good managers answer these explicitly; bad managers answer them through six rounds of "no, not like that." AI systems are the limit case of the junior employee: enormously capable within the frame you give them, and entirely unable to infer the frame you didn't. An LLM is a reasoning and generation engine. It will fill every gap in your specification with a plausible guess, delivered fluently. The previous lessons named this pattern — false confidence — and the cure is not a better model. The cure is fewer gaps.

There is also an organizational reason, which matters for anyone building an AI-native company rather than a pile of automations. A written spec is the unit of governance. You cannot evaluate a workflow without acceptance criteria. You cannot audit a workflow without knowing its allowed inputs. You cannot bound an agent's permissions without written non-goals. Everything in the governance stack — evaluations, audit, approval gates — hangs off the spec. No spec, no governance; just vibes with a cron schedule.

3. Anatomy of the Spec, Front Half

Today's material is five components. The first two define *purpose*: objective and non-goals. The next two define *interface*: allowed inputs and the output contract. The fifth — acceptance criteria — binds the interface back to the purpose. We take them in order.

3.1 The Objective: One Sentence, Aimed at an Outcome

The objective is a single sentence stating the business outcome the workflow exists to produce. Not the activity — the outcome.

This distinction does most of the work, so it deserves care. Consider two candidate objectives for a business development workflow:

Activity-shaped: "Draft outreach emails to potential Galoy partners."

>

Outcome-shaped: "Each week, produce a ranked list of 10–20 banks and fintechs that are credible near-term prospects for Galoy's Bitcoin banking infrastructure, with enough verified context on each that a human can decide whether to reach out."

The first sentence describes what the workflow *does*. The second describes what it is *for*, and everything downstream follows from it. The outcome-shaped objective tells you the output is a ranked list, not emails. It tells you the consumer is a human making a decision, which means the output must support judgment rather than replace it. It implies quality bars: "credible near-term," "verified context." And it quietly rules things out — this workflow informs outreach; it does not perform outreach.

A useful test: **could the objective be false?** "Draft outreach emails" can't fail — any emails satisfy it. "Produce a ranked list a human can act on" can fail in observable ways: the list could be unranked, the targets implausible, the context unverified. An objective that cannot be false cannot be evaluated, and a workflow that cannot be evaluated cannot be governed. If your objective is compatible with every possible output, you haven't written an objective; you've written a category.

Practical constraints on the objective:

- **One sentence.** If you need two, you have two workflows. Split them. Small, focused workflows are easier to specify, evaluate, and debug — this is also the argument behind Factor 10 of the 12-Factor Agents material in the reading guide.
- **Name the consumer.** Who or what receives the output? A human deciding? A downstream system parsing? The consumer determines the contract.
- **Name the cadence if there is one.** "Each week" is part of the objective, not a scheduling detail. A weekly target list and an on-demand target list are different workflows with different freshness requirements.

3.2 Non-Goals: Decisions, Not Omissions

Non-goals are the things the workflow must not do or attempt — written down, deliberately.

This is the component operators most often skip, on the theory that anything not mentioned is obviously out of scope. That theory is wrong for two separate reasons.

First, the model reason: an LLM does not distinguish between "unmentioned because forbidden" and "unmentioned because obvious." Given the objective "produce a ranked BD target list," a capable model asked to be *helpful* may draft the outreach emails too, or infer contact details for named executives, or editorialize about which competitor to undercut. Each of these is a plausible, helpful-seeming extension of the objective. Each is also an action nobody authorized. The failure-modes lessons called this category unsafe action; the non-goals section is where you prevent it at the specification layer, before you have runtime gates to enforce it.

Second, the organizational reason: unwritten scope boundaries erode. Every incremental prompt edit — "also add a suggested opening line" — is individually reasonable, and cumulatively they turn a research workflow into an outreach workflow with no moment at which anyone decided that. Drift, as covered previously, is rarely one bad change; it is thirty defensible ones. A written non-goals list gives every future edit something to collide with. "Also send the emails" stops being an incremental improvement and becomes what it actually is: a violation of a written decision, requiring a new decision.

Non-goals for the Galoy BD workflow might read:

Non-goals. This workflow does not: contact anyone or send anything; guess contact information not found in cited sources; assess targets' financial health beyond what public sources state; produce outreach copy; maintain state about which targets were previously contacted (that lives in the CRM, not here).

Notice the texture. Each non-goal is specific enough to check, and several encode real judgments — the decision that contact-state belongs to the CRM is an architectural decision about where memory lives, recorded in one clause. Good non-goals read like the minutes of arguments you decided not to have twice.

A discipline note: non-goals are not a disclaimer wall. Five sharp exclusions beat twenty defensive ones. If you find yourself writing "this workflow does not achieve world peace," you have stopped making decisions and started padding.

3.3 Allowed Inputs: The Guest List

The inputs section enumerates what the workflow may consume — and by enumeration, defines everything it may not. The stance to internalize: **inputs are allowlisted, not open**. Anything not named is not an input, no matter how relevant it looks at runtime.

This is the direct spec-layer answer to Workflow Failure Modes I. A workflow that consumed a stale dataset, an untrusted web page, or a colleague's half-finished draft did not malfunction — it did exactly what an unspecified input policy permits, which is anything. For each allowed input, the spec answers three questions:

1. **Source.** Where does it come from, exactly? "The web" is not a source. "Company press pages, financial-press coverage, regulatory filings, and conference speaker lists" is a source list.
2. **Freshness.** How old may it be before it is no longer valid input? A BD target list built on an eighteen-month-old funding announcement is worse than no list, because it arrives wearing the costume of research.
3. **Trust level.** Is this input authoritative, indicative, or merely suggestive? A regulatory filing and a Reddit thread can both mention the same bank; the spec should say the workflow may cite the first as fact and the second, at most, as a lead to verify.

For the Galoy example:

Allowed inputs. (a) The current target-thesis memo (internal, authoritative, must be the latest version); (b) public news and press coverage of banks and fintechs, no older than 90 days for ranking signals; (c) regulatory and licensing databases, authoritative; (d) conference agendas and speaker lists, indicative only. **Not inputs:** private correspondence, prior outreach threads, paywalled reports we haven't licensed, anything the workflow cannot cite.

That last clause — *anything the workflow cannot cite* — is doing quiet, heavy work. It couples the input policy to the output contract: if a claim in the output can't be traced to an allowed input, the claim is invalid regardless of whether it happens to be true. This is how you make false confidence structurally detectable rather than relying on someone's suspicion.

One sentence on an adjacent topic, then back to the thread: in a retrieval-backed system, the allowed-inputs section is effectively the retrieval scope — the memory layer you built in the Operating Memory unit serves whatever this section permits, and nothing else.

3.4 The Output Contract: Shape Before Content

The output contract specifies what the workflow produces: its format, its required fields, and the guarantees each field carries. The right mental model comes from software: a typed interface. Downstream consumers — a human reader, a dashboard, another workflow — should be able to rely on the output's *shape* without inspecting each instance, the same way calling code relies on a function's signature without reading its body.

The failure this prevents is subtle because it is not a quality failure — it is a *variability* failure. Ask an unconstrained model for a research summary on Monday and you get five paragraphs; on Tuesday, a bulleted list; on Wednesday, five paragraphs plus an unsolicited recommendation. Each output might be individually fine. But nothing can be built downstream of an output whose shape is a mood. Composition — workflows feeding workflows, which is the entire premise of an AI-native operating layer — requires contracts at every seam.

A contract for the Galoy BD workflow's output:

Output contract. A Markdown document containing: (1) a ranked table of 10–20 targets, each row carrying company name, segment, one-line relevance thesis, a rank score with the reason for the score, and citations for every factual claim; (2) a "removed since last week" section listing any previously ranked target that dropped out, with the reason; (3) a "confidence and gaps" section stating what the workflow could not verify. Every factual claim cites an allowed input. No field may be silently omitted; if a field cannot be filled, it contains UNVERIFIED and the gap appears in section 3.

Three design principles are embedded there, and they generalize:

- **The contract includes provenance.** Citations are not garnish; they are a required field. An uncited claim is a contract violation even when it is correct — because you cannot govern a workflow whose validity depends on spot-checking its facts.
- **The contract includes the workflow's own uncertainty.** The "confidence and gaps" section forces the system to surface what it doesn't know, structurally. This is the standing antidote to false confidence: don't hope the model volunteers doubt; make doubt a required field and its absence a detectable violation.
- **Silence is forbidden.** The most dangerous outputs are missing fields that nobody notices are missing. UNVERIFIED is ugly on purpose. Ugly and visible beats clean and wrong.

The 12-Factor Agents material sharpens this idea from the engineering side: treat model outputs as structured data crossing a defined boundary, not as prose to be eyeballed. You will meet that framing in the reading guide; the spec-level version is simply — *write the schema down before you generate anything against it*.

3.5 Acceptance Criteria: How You'd Know

Acceptance criteria are the checks that determine whether one specific output honors the contract and serves the objective. They are the hinge between the two halves of the spec: written today as part of the interface, but operated at runtime — feeding the gates and the definition of done covered in Part II. Later still, they become the seed of your evaluation suite; automated evaluations are just acceptance criteria that learned to run themselves.

The bar for a well-formed acceptance criterion: **a competent person who did not build the workflow can apply it to a single output and reach a yes/no answer**. Three corollaries fall out of that sentence. It must be checkable per-output (not "the list is generally good" but "every rank score has a stated reason"). It must not require the author's private context (if only you can judge it, it is a taste, not a criterion). And it must be binary at the point of decision — thresholds are fine, vibes are not.

For the Galoy workflow:

Acceptance criteria. (1) Between 10 and 20 targets; count outside range → reject. (2) Every factual claim carries a citation resolving to an allowed input; one uncited claim → reject. (3) No ranking signal older than 90 days. (4) No target appears that matches a non-goal exclusion (e.g., contact info the sources didn't provide). (5) The gaps section is present and non-empty — a research workflow reporting zero uncertainty is presumptively wrong, not presumptively excellent. (6) Spot-check two citations at random; either misrepresents its source → reject the whole output.

Criterion 5 deserves a highlight because it inverts a common instinct. In most quality systems, flaws are bad. Here, a *reported absence of flaws* is the red flag — the previous lessons taught why. Encode that lesson as a criterion and it stops depending on anyone's vigilance.

4. The Spec at a Glance

Component	Question it answers	Failure it prevents	Galoy BD fragment
Objective	What outcome, for whom, how often?	Drift; unevaluable output	Weekly ranked list of 10–20 credible prospects a human can act on
Non-goals	What must this never do or attempt?	Unsafe action; scope creep	No contact, no guessed emails, no outreach copy
Allowed inputs	What may it consume, how fresh, how trusted?	Bad inputs; unverifiable claims	Thesis memo + public sources ≤ 90 days; nothing uncitable
Output contract	What shape and guarantees does output carry?	Uncomposable, silently incomplete output	Ranked table + removals + gaps; UNVERIFIED over silence
Acceptance criteria	How would a third party check one output?	False confidence surviving review	Citations resolve; gaps section non-empty; random spot-check

Five rows. If you can fill this table for a workflow, you understand the workflow. If you can't, you are not ready to automate it — and no model choice will fix that.

5. Two Compact Worked Examples

The Galoy example ran through the whole paper. Here are two more workflows from the required set, compressed to show that a real spec's front half fits in under a page. Brevity is a feature: a spec nobody rereads is a spec nobody enforces.

5.1 The Daily Brief

Objective. Every morning by 07:00, produce a one-page brief that lets the operator start the day already oriented: what changed overnight in active projects, markets relevant to current theses, and hard commitments in the next 48 hours.

>

Non-goals. Does not recommend decisions. Does not summarize things that didn't change ("no update" is one line, not a paragraph). Does not pull from personal correspondence. Does not editorialize on market moves.

>

Allowed inputs. Calendar (next 48h); project-state notes updated within 7 days; a fixed, named list of news sources for overnight coverage; market data from the designated feed only.

>

Output contract. Max one page. Three fixed sections: *Changed*, *Unchanged but time-sensitive*, *Next 48 hours*. Every item timestamped and sourced. Empty section renders as "Nothing," not as filler.

>

Acceptance criteria. Fits one page; every item traceable to an allowed input with a timestamp; no item in *Changed* older than 24 hours; no advice verbs ("should," "consider") anywhere.

Note the dog that doesn't bark: the non-goal "*no update is one line*." A daily workflow's characteristic failure is padding — the system generating content because generating content is what it does. The spec makes emptiness a valid, even honorable, output. This is harder than it sounds and worth stealing.

5.2 The Research Packet

Objective. On request, produce a packet on a named company or person that makes a 30-minute meeting worth having: who they are, what they've publicly said and built, and where their interests plausibly intersect ours.

>

Non-goals. No speculation about private finances or internal politics. No sentiment scoring of the subject. No inclusion of any claim traceable only to anonymous sources. Does not decide whether the meeting should happen.

>

Allowed inputs. The subject's public body of work — talks, writing, filings, interviews, code — plus reputable press. Public-content only, by design; this is the same wedge as the public-content knowledge-base work, and the input policy *is* the ethical boundary of the product.

>

Output contract. Fixed sections: *Background* (facts, cited), *Public positions* (direct quotes with dates and links — quotes, not paraphrases), *Intersection points* (each explicitly labeled as inference, with the facts it rests on), *Gaps*. Facts and inferences never share a section.

>

Acceptance criteria. Every quote verbatim and linked; every intersection point labeled as inference and grounded in cited facts; gaps section non-empty; nothing violating the anonymity non-goal.

The load-bearing decision here is the fact/inference separation in the contract. Research products fail when informed guesses launder themselves into facts through confident prose. You cannot prompt that risk away reliably, but you can make it a structural violation: an inference appearing in the *Background* section is checkably wrong regardless of whether it is true.

6. What the Spec Cannot Do: The Honest Limitation

A spec is necessary. It is nowhere near sufficient, and pretending otherwise is its own failure mode. Three limitations, in ascending order of importance.

First, a spec does not enforce itself. Everything in this paper is prose. A model can violate every clause of it fluently, and on a long enough timeline it will. Enforcement is a runtime concern — gates that check acceptance criteria before output ships, permissions that make non-goals physically impossible rather than merely forbidden, and human approval where the stakes warrant it. That is Part II. Until then, hold the honest frame: today you learned to write the law; next lesson you build the courts.

Second, specs can be theater. There is a failure mode where the spec is written *after* the automation, reverse-engineered from whatever the system already does, so that every output passes by construction. Such a spec has the form of governance and none of the substance. The tell is history: a real spec predates the workflow and occasionally *rejects* outputs. A spec that has never rejected anything is either guarding a perfect system or performing a ritual, and you know which is more likely.

Third — and this is the deep one — the spec is only as good as your understanding of the workflow. Writing a spec for a process you don't actually understand produces a precise description of your misunderstanding, which the automation will then execute at scale and with confidence. This is why the exercise below asks you to spec a workflow you have personally done by hand. The spec is a compression of operational knowledge; you cannot compress what you don't possess. When the spec-writing feels hard — when you can't state the objective in one sentence, or can't decide whether something is an input — that difficulty is not friction in the method. It is the method, telling you where your understanding of your own operation is thinner than you assumed. Cheapest diagnostic you will ever run.

7. Application: One Spec, One Page, One Workflow

The exercise is deliberately small. Pick **one** workflow you currently perform by hand and have performed at least three times — a recurring brief, a research task, a reporting step. Write the front half of its spec: objective (one sentence), non-goals (three to five), allowed inputs (with source, freshness, trust), output contract (sections and required fields), and three acceptance criteria a stranger could apply.

One page, hard limit. Do not automate anything yet. Do not build anything around it. The artifact is the page; the lesson is what writing the page forces you to notice. Keep it — Part II extends this same document with gates and failure handling, and a spec you wrote this week is far better raw material than one invented on the spot next week.

8. Vocabulary

- **Workflow spec** — A written document, preceding automation, that defines a workflow's purpose, boundaries, interface, and checks. Independent of any particular model or tool.
- **Objective** — One sentence stating the business outcome the workflow exists to produce, including its consumer and cadence. Must be falsifiable: some outputs must be able to fail it.
- **Non-goals** — Explicit, deliberate exclusions: what the workflow must not do or attempt. Decisions, not omissions; the spec-layer precursor to runtime permissions.
- **Allowed inputs** — The enumerated set of sources a workflow may consume, each with source, freshness requirement, and trust level. Allowlisted: anything unnamed is excluded.
- **Output contract** — The specified shape of the workflow's output: format, required fields, and the guarantees (provenance, stated uncertainty) each field carries. The seam that makes workflows composable.

- **Acceptance criteria** — Per-output, third-party-checkable, yes/no tests that an output honors the contract and serves the objective. The seed of a future evaluation suite.
- **Spec theater** — A spec reverse-engineered from existing behavior so that everything passes by construction; governance in form, not substance.
- **Provenance** — The traceable origin of a claim in an output; in a well-formed contract, a required field rather than a courtesy.

9. Reading Guide

12-Factor Agents — Dex Horthy / HumanLayer —
[\[github.com/humanlayer/12-factor-agents\]](https://github.com/humanlayer/12-factor-agents)(<https://github.com/humanlayer/12-factor-agents>)

A practitioner's manifesto on building LLM-powered software that actually ships, structured as twelve factors in the spirit of the original twelve-factor app. Read the introduction and skim all twelve factor titles first (about ten minutes) — the framing that reliable agents are "mostly deterministic software with LLM steps in the right places" is the engineering-side twin of this paper's thesis.

Then read three factors closely, mapped to today's concepts:

- **Factor 4 — Tools are structured outputs.** The engineering realization of the output contract: model output as structured data crossing a defined boundary, not prose to be eyeballed. If today's contract idea felt abstract, this makes it concrete.
- **Factor 3 — Own your context window.** The runtime counterpart of allowed inputs: deciding deliberately what the model sees, rather than letting a framework decide for you.
- **Factor 10 — Small, focused agents.** The case for one-sentence objectives, argued from reliability: scope you can specify is scope you can trust.

Read as an operator, not an implementer — for each factor, ask "what would the spec have to say for a team to build this correctly?" Total time: 30–40 minutes. Skip the code samples without guilt; the ideas survive the skipping.

10. Walking Questions

1. Take a workflow you ran by hand this week. Can you state its objective in one falsifiable sentence — and who is the consumer of its output?
2. Which of your current automations has *zero written non-goals*? What is the most plausible unsafe or scope-creeping thing it could do tomorrow while technically following its instructions?
3. Where in your own operation does a workflow's output currently vary in shape from run to run — and what downstream thing does that variability quietly prevent you from building?
4. For the Galoy-style BD list: which acceptance criterion would you add that this paper missed? (There is at least one obvious candidate around duplicate targets across weeks.)
5. Which part of today's five components could you teach to someone else right now, and which would you have to reread — and what does that split tell you?

One-Sentence Takeaway

Before you automate a workflow, write down what it's for, what it must never do, what it may consume, and what its output must look like — because an AI system will fill every gap you leave with a confident guess, and a one-page spec is the cheapest place you will ever catch it.

Next in this unit — Writing the Workflow Spec II: Gates, Failure Handling, and Done. Today you wrote the law; next time you build the courts: the runtime checks that enforce acceptance criteria, what the workflow does when they fail, and how it knows it is finished.